


RHEINLAND-PFÄLZISCHE TECHNISCHE UNIVERSITÄT
KAISERSLAUTERN-LANDAU

BACHELOR'S THESIS

Reinforcement Learning for Group Contribution Methods

Author:
Omar HUMEEDAT

Supervisor:
Prof. Dr. Sophie FELLEENZ
Ms. WeiChen Li
Mr. Billy Franks

*A thesis submitted in fulfillment of the requirements
for the degree of Bachelor of Science*

in the

Machine Learning Group
Department of Computer Science

March 11, 2025

Declaration of Authorship

I, Omar HUMEEDAT, declare that this thesis titled, “Reinforcement Learning for Group Contribution Methods” and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Date:

RHEINLAND-PFÄLZISCHE TECHNISCHE UNIVERSITÄT
KAISERSLAUTERN-LANDAU

Abstract

Faculty Name
Department of Computer Science

Bachelor of Science

Reinforcement Learning for Group Contribution Methods

by Omar HUMEEDAT

Diese Arbeit untersucht die Anwendung von Reinforcement Learning (RL) zur Merkmalsauswahl bei der Vorhersage des Dampfdrucks unter Verwendung des SIMPOL.1-Modells. Traditionelle Methoden zur Merkmalsauswahl, wie Lasso, Recursive Feature Elimination (RFE) und genetische Algorithmen (GA), stoßen häufig in hochdimensionalen Räumen an ihre Grenzen, in denen Beziehungen zwischen Merkmalen komplex und nicht-linear sein können. Um dieses Problem zu lösen, wurde ein RL-basierter Ansatz unter Verwendung von Deep Q-Networks (DQN) und Dueling DQN eingesetzt, um die Merkmalsauswahl dynamisch zu optimieren und die Vorhersagegenauigkeit des SIMPOL.1-Modells zu verbessern.

Zwei Datensätze, die Dortmund-Datenbank (DDB) und der DIPPR-Datensatz, wurden verwendet, um die Leistung der Modelle zu bewerten. Die RL-basierten Modelle zeigten eine überlegene Genauigkeit und Merkmalsreduktion im Vergleich zu traditionellen Methoden. So reduzierte das Dueling DQN die Anzahl der ausgewählten Merkmale und erzielte dabei geringere Werte für den Mean Squared Error (MSE) und den Mean Absolute Error (MAE) als die traditionellen Methoden. Darüber hinaus ermöglichte die Upper Confidence Bound (UCB)-Explorationsstrategie eine schnellere Konvergenz und bessere Merkmalsauswahl.

Zudem führte der Einsatz von GPU-Beschleunigung, Caching und Inverse Reinforcement Learning (IRL) für den DDB-Datensatz zu einer signifikanten Reduzierung der Trainingszeit. Konkret konnte die Trainingszeit für die DDB von 32 Stunden auf 5 Stunden und für den DIPPR-Datensatz von 8 Stunden auf 2,5 Stunden reduziert werden, während IRL sich als effektiv bei der Beschleunigung der SIMPOL.1-Bewertung während des Agententrainings erwies.

Diese Arbeit zeigt, dass die RL-basierte Merkmalsauswahl erhebliche Vorteile bei der Handhabung hochdimensionaler Daten bietet, indem sowohl die Genauigkeit verbessert als auch die Rechenleistung optimiert wird. Die Ergebnisse deuten darauf hin, dass RL traditionelle Methoden der Merkmalsauswahl bei Vorhersageaufgaben übertreffen kann und vielversprechende Anwendungen in Bereichen wie der Chemieingenieurtechnik und darüber hinaus bietet.

Acknowledgements

I would like to express my deepest gratitude to Prof. Dr. Sophie Fellenz for giving me the opportunity to delve deeper into the field of Reinforcement Learning. A special thanks to my supervisors Ms. WeiChen Li and Mr. Billy Franks, for their constant support and insightful feedback, which greatly contributed to the success of this work.

I would also like to thank my mother for her unwavering support and encouragement during this challenging journey. Finally, I dedicate this thesis to the memory of my father, who passed away before he could see me graduate.

Contents

Declaration of Authorship	ii
Abstract	iii
Acknowledgements	iv
1 Introduction	1
2 Backgrounds	3
2.1 Vapor Pressure	3
2.2 Functional Groups and Their Role in Vapor Pressure	3
2.3 Group Contribution Method (GCM)	3
2.3.1 SIMPOL.1 Model	4
2.4 Feature Selection in Machine Learning	5
2.5 Reinforcement Learning (RL)	5
2.5.1 Markov Decision Process (MDP)	6
2.5.2 Q-Learning	6
2.5.3 Deep Q-Networks (DQN)	6
3 Literature Review	7
3.1 Predicting Thermodynamic Properties	7
3.2 Reinforcement Learning for Feature Selection	7
3.3 Markov Decision Process (MDP) for Feature Selection	8
3.4 Deep Q-Network (DQN) for High-Dimensional Feature Selection	8
3.5 Q-Learning with Upper Confidence Bound (UCB) Exploration	8
3.6 Inverse Reinforcement Learning (IRL) for Learning from Historical Data . . .	9
4 Methodology	10
4.1 Datasets and Preprocessing	10
4.1.1 Datasets Overview	10
4.1.2 Feature Extraction	10
4.1.3 Data Splitting	11
4.1.4 Data Normalization	11
4.2 Reinforcement Learning Approach for Feature Selection	11
4.2.1 Problem Formulation	11
4.2.2 Environment	11
State Representation	12
Action Space	12
4.2.3 Action Masking and Search Space Reduction	12
Reward Function	13
4.2.4 Why Start with All Features?	13
4.2.5 Agent: Deep Q-Network (DQN)	14
Standard DQN Model	14

	Dueling DQN Model	14
4.2.6	Target Network and Soft Updates	15
4.2.7	Exploration Strategies	15
4.2.8	Replay Buffer	16
4.3	Optimization	16
4.4	Experimental Setup	16
4.4.1	Training Phases	17
4.4.2	Evaluation Metrics	17
5	Results	19
5.1	Traditional Feature Selection Methods Performance	19
5.2	RL-Based Feature Selection Performance	19
5.3	Impact of Exploration Strategy	20
5.4	Computational Efficiency	20
6	Discussion	22
6.0.1	Impact on Prediction Accuracy	22
6.0.2	Exploration Strategies	22
6.0.3	Computational Efficiency	22
6.0.4	Limitations and Future Work	23
7	Conclusion	24
	Bibliography	25

List of Figures

2.1	Diagram of the ethanol molecule (C_2H_5OH) indicating the functional groups: the hydroxyl (-OH) group denoting its classification as an alcohol and the ethyl (C_2H_5) group contributing to its overall molecular behavior.	4
4.1	reinforcement learning approach	12
4.2	Feature Selection Mechanisem	13
5.1	Convergence of RL Models using Epsilon-Greedy and UCB Strategies for DDB and DIPPR Datasets	20

List of Tables

4.1	Raw Data: Temperature, Pressure, and Molecular Information	10
4.2	Processed Data: Extracted Features for Each Molecule Based on Temperature (T) from Raw Data.	11
4.3	Table of Functional Groups and Their Corresponding Names	18
5.1	Performance of Traditional Feature Selection Methods for Both DDB and DIPPR Datasets for enhancing SIMPOL.1 method's performance	19
5.2	Performance of RL-Based Feature Selection Methods for Both DDB and DIPPR Datasets	19
5.3	Training Time with and without Acceleration for DDB and DIPPR Datasets .	20

Chapter 1

Introduction

In the field of chemical engineering, accurately predicting thermodynamic properties such as **vapor pressure** is critical for optimizing various industrial processes. Vapor pressure plays an essential role in operations like distillation, evaporation, and chemical reactions, making its precise prediction vital for ensuring safety, efficiency, and environmental sustainability. The ability to predict vapor pressure with high accuracy can significantly improve process control in industries such as pharmaceuticals, petrochemicals, and environmental engineering. However, vapor pressure prediction is not a straightforward task, as it depends heavily on the molecular structure of the compounds involved [7].

A molecule's structure is composed of **functional groups**, which are specific arrangements of atoms that determine the chemical behavior and physical properties of the compound. Functional groups significantly influence vapor pressure, as they govern the interactions and phase changes within a molecule. Traditionally, models based on the **Group Contribution Method (GCM)** have been used to estimate properties like vapor pressure by summing the contributions of individual functional groups. One widely used model that implements this principle is **SIMPOL.1**, which predicts vapor pressure by aggregating the contributions of functional groups present in the molecular structure. While these models have proven to be effective for many applications, they also face limitations, particularly in cases where not all functional groups are equally relevant [12].

A major challenge in vapor pressure prediction is that not every functional group in a molecule has the same influence on vapor pressure. Some groups may contribute significantly, while others may have little or no effect. Including too many irrelevant or redundant functional groups can complicate the model, introducing noise and reducing prediction accuracy. Therefore, the central question arises: *How can we efficiently identify the most relevant functional groups to improve the accuracy of vapor pressure predictions?*

This problem can be viewed as one of **feature selection**, a key step in building predictive models. In machine learning, feature selection refers to the process of selecting the most important variables (features) from a larger dataset to enhance model performance. By focusing on the most relevant features, a model can reduce complexity, improve accuracy, and increase computational efficiency [1]. In the context of vapor pressure prediction, the features to be selected are the functional groups that most strongly influence vapor pressure. While there are traditional methods for feature selection, such as **Lasso regression**, **Recursive Feature Elimination (RFE)**, and **Genetic Algorithms**, which have been applied successfully across various fields, these approaches may not always capture the complex relationships between functional groups and vapor pressure. The detailed discussion of these traditional methods will be covered in the next chapter. However, this thesis aims to explore whether more dynamic and adaptive methods, like **reinforcement learning (RL)**, can offer a more effective solution for feature selection in the specific context of vapor pressure prediction.

Reinforcement learning (RL) is a machine learning technique that enables an agent to interact with an environment, learn from feedback, and improve its decision-making over

time [16]. In RL, the agent explores various actions, receives rewards or penalties based on its performance, and gradually learns an optimal strategy to maximize cumulative rewards. In the context of feature selection for vapor pressure prediction, reinforcement learning can dynamically evaluate different combinations of functional groups and determine which groups contribute most significantly to the model's accuracy. The RL agent learns through trial and error, refining its selection strategy over time and potentially uncovering complex, non-linear relationships between functional groups that traditional methods might miss.

The potential of reinforcement learning lies in its ability to adapt to the complexities of the problem, unlike static traditional methods. However, it is important to note that this thesis does not assume that reinforcement learning is the definitive solution to the feature selection problem. Rather, the objective of this thesis is to investigate whether RL can offer improvements in accuracy and efficiency compared to traditional methods, specifically in the context of vapor pressure prediction. The goal is to evaluate whether reinforcement learning can help address the challenge of selecting the most relevant functional groups from large datasets and whether it outperforms traditional approaches in terms of predictive power and computational costs.

This thesis aims to contribute to the broader understanding of how machine learning, and specifically reinforcement learning, can be applied to feature selection in chemical engineering. Vapor pressure prediction is a critical aspect of many industrial applications, and improving the accuracy of these predictions could lead to more efficient and cost-effective processes. By comparing reinforcement learning to traditional feature selection methods, I seek in this thesis to provide insights into the effectiveness of RL in handling high-dimensional molecular data and determining the functional groups that matter most for accurate vapor pressure predictions.

The structure of this thesis is organized as follows: **Chapter 2: Background** will delve into the theoretical concepts underlying vapor pressure prediction, the Group Contribution Method, traditional feature selection techniques, and an introduction to reinforcement learning. **Chapter 3: Literature Review** presents a review of some previous work in feature selection, particularly focusing on the application of reinforcement learning in feature selection. It discusses relevant studies and highlights key RL techniques such as Deep Q-Networks (DQN), UCB as exploration strategy and Inverse Reinforcement Learning (IRL). **Chapter 4: Methodology** outlines the approach used to implement reinforcement learning for feature selection, including data preparation, model design, and the experimental framework. **Chapter 5: Results** presents the outcomes of the experiments comparing RL to traditional feature selection methods, focusing on the accuracy and efficiency of the models. **Chapter 6: Discussion** interprets the results in relation to the research questions and explores the practical implications of using RL for feature selection in vapor pressure prediction. Finally, **Chapter 7: Conclusion** summarizes the key findings and suggests future directions for further research and improvements in this area.

Chapter 2

Backgrounds

2.1 Vapor Pressure

Vapor pressure is a thermodynamic property that measures the pressure exerted by a vapor in equilibrium with its liquid (or solid) phase at a given temperature. The higher the vapor pressure of a substance, the more volatile it is. As temperature increases, more molecules gain enough kinetic energy to escape from the liquid phase into the vapor phase, causing vapor pressure to rise [15].

Vapor pressure is used in many industrial applications, including chemical engineering and environmental science. For instance, in distillation processes, vapor pressure data helps separate volatile compounds. Similarly, vapor pressure is important for predicting the behavior of volatile organic compounds (VOCs) in the atmosphere, where evaporation and atmospheric dispersion affect pollution levels [15].

Predicting vapor pressure accurately is challenging due to the variety of chemical compounds and the complex role that molecular structure plays. Functional groups in chemical compounds, such as hydroxyl (-OH) or methyl (-CH₃) groups, have a significant impact on vapor pressure [14, 15]. However, the relationships between functional groups and vapor pressure are often non-linear and difficult to model, particularly for multifunctional organic compounds. This complexity necessitates computational methods like the Group Contribution Method (GCM) to estimate vapor pressure based on molecular structure.

2.2 Functional Groups and Their Role in Vapor Pressure

Functional groups play a critical role in determining a molecule's chemical behavior, including its vapor pressure, which varies with temperature. Hydroxyl groups (-OH) lower vapor pressure due to the formation of hydrogen bonds, which require more energy to break. As temperature rises, molecules with hydroxyl groups exhibit slower increases in vapor pressure because these bonds resist vaporization [7].

In contrast, methyl groups (-CH₃) lead to higher vapor pressures due to weaker van der Waals forces. As temperature increases, the vapor pressure of molecules with methyl groups rises more rapidly, as these weaker forces are easier to overcome [7].

The impact of functional groups on vapor pressure is not uniform [12], and identifying the most significant groups at various temperatures is challenging. The complex, non-linear relationships between functional groups, molecular structure, and temperature require sophisticated models to predict vapor pressure accurately.

2.3 Group Contribution Method (GCM)

The **Group Contribution Method (GCM)** is a widely used approach for estimating the thermodynamic properties of molecules, including vapor pressure. This method simplifies

the prediction task by breaking down a molecule into its constituent functional groups and assigning each group a specific contribution to the property being predicted. The total property of the molecule is then estimated as the sum of the contributions from all functional groups [7].

Mathematically, the GCM can be expressed as:

$$P = \sum_i v_i \cdot f_i$$

Where:

- P is the predicted property (e.g., vapor pressure),
- v_i is the number of occurrences of functional group i ,
- f_i is the contribution of functional group i to the property.

This method is particularly useful in situations where direct experimental measurement of a property, such as vapor pressure, is difficult or impractical. One of the most prominent models that uses GCM is the **SIMPOL.1** model, which was developed to estimate vapor pressures and enthalpies of vaporization for multifunctional organic compounds [12].

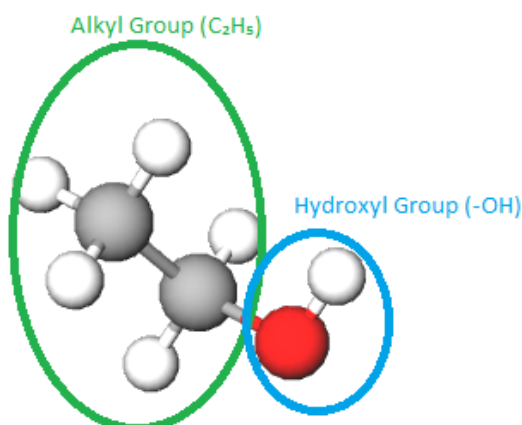


FIGURE 2.1: Diagram of the ethanol molecule (C_2H_5OH) indicating the functional groups: the hydroxyl (-OH) group denoting its classification as an alcohol and the ethyl (C_2H_5) group contributing to its overall molecular behavior.

2.3.1 SIMPOL.1 Model

The **SIMPOL.1** model, developed by Pankow and Asher in 2008, is a simple group contribution method specifically designed for predicting the vapor pressures of multifunctional organic compounds. SIMPOL.1 is especially relevant for environmental studies, as it is used to estimate the vapor pressures of volatile organic compounds involved in atmospheric chemistry [12].

The model expresses the logarithm of the vapor pressure of a compound at a given temperature as the sum of contributions from individual functional groups within the molecule. This can be written as:

$$\log_{10} p_{L,i}^0(T) = \sum_k v_{k,i} b_k(T) \quad (2.1)$$

Where:

- $p_{L,i}^o(T)$ is the vapor pressure of compound i at temperature T ,
- $v_{k,i}$ is the number of occurrences of group k in compound i ,
- $b_k(T)$ is the temperature-dependent contribution of group k .

The contribution of each group $b_k(T)$ is calculated using a temperature-dependent polynomial function:

$$b_k(T) = \frac{B_{1,k}}{T} + B_{2,k} + B_{3,k}T + B_{4,k} \ln T \quad (2.2)$$

Where $B_{1,k}, B_{2,k}, B_{3,k}, B_{4,k}$ are empirically determined coefficients specific to each functional group [12].

While the SIMPOL.1 model is effective for predicting vapor pressures, it has limitations. The model assumes that each functional group contributes independently to vapor pressure, which may not always be true, especially in complex molecules. Additionally, the model may include redundant or irrelevant functional groups, reducing the efficiency and accuracy of the predictions. This limitation leads to the need for **feature selection** to improve the performance of the model.

2.4 Feature Selection in Machine Learning

Feature selection is the process of identifying the most relevant variables (features) for a predictive model while removing those that are redundant or irrelevant. In high-dimensional datasets, feature selection is crucial for improving the accuracy of models and reducing computational complexity. When dealing with molecular data, where each molecule can have tens or hundreds of potential functional groups, feature selection becomes essential to identify the groups that most significantly affect the target property, in this case, vapor pressure.

There are several common approaches to feature selection [6, 3, 10]:

Filter Methods: These methods evaluate the relevance of features by calculating statistical properties such as correlation with the target variable. Filter methods are computationally efficient but may overlook important interactions between features.

Wrapper Methods: Wrapper methods, such as **Recursive Feature Elimination (RFE)**, evaluate subsets of features by training and testing the model on different combinations. These methods often produce more accurate results than filter methods but are computationally expensive.

Embedded Methods: Embedded methods integrate feature selection into the model training process. For example, **Lasso regression** performs feature selection by shrinking the coefficients of irrelevant features to zero. Embedded methods are efficient and can handle some interactions between features, but they may not be sufficient for highly non-linear relationships.

Despite their utility, traditional feature selection methods often struggle with non-linear relationships and complex interactions between features, particularly in high-dimensional datasets. This is where more advanced techniques, such as **Reinforcement Learning (RL)**, can be advantageous.

2.5 Reinforcement Learning (RL)

Reinforcement Learning (RL) is a machine learning approach where an agent learns to make decisions through interaction with an environment. The agent’s goal is to maximize the

cumulative reward over time by learning a policy is a mapping from states to actions. In the context of feature selection, RL can be used to explore different subsets of features and receive feedback on their impact on the model's performance [9, 16].

2.5.1 Markov Decision Process (MDP)

RL problems are often framed as **Markov Decision Processes (MDP)**. An MDP consists of the following elements [9]:

- **States (S):** The current configuration of the environment. In feature selection, this could represent the current subset of selected features.
- **Actions (A):** The decisions the agent can take. In feature selection, the actions are selecting or deselecting a feature.
- **Rewards (R):** The feedback the agent receives after taking an action, which indicates the quality of the action. In feature selection, the reward will be based on the improvement or decline in model performance.
- **Policy (π):** A strategy that the agent uses to decide which actions to take based on the current state.

The agent's objective is to learn an optimal policy that maximizes long-term rewards by iterating over multiple steps of state-action-reward sequences.

2.5.2 Q-Learning

One of the most popular RL algorithms is **Q-learning**, a model-free method where the agent learns to estimate the value of taking an action in a particular state, denoted as the Q-value. The goal of Q-learning is to learn the optimal Q-function, $Q^*(s, a)$, which estimates the maximum expected future reward for each state-action pair [20].

Q-values are updated using the following equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (2.3)$$

Where:

- s_t and a_t are the state and action at time t ,
- r_{t+1} is the reward received after taking action a_t ,
- α is the learning rate, and
- γ is the discount factor, which determines the importance of future rewards.

2.5.3 Deep Q-Networks (DQN)

For problems with large state-action spaces, **Deep Q-Networks (DQN)** are used to approximate the Q-function with a neural network. DQN replaces the traditional Q-value table with a neural network that outputs Q-values for each action given the current state [11, 17]. This is particularly useful in high-dimensional feature selection tasks where the space of possible feature subsets for N features is 2^N permutations.

Chapter 3

Literature Review

With the increasing complexity of chemical systems and the availability of large datasets, traditional methods for predicting thermodynamic properties, such as vapor pressure, have demonstrated limitations. Historically, methods such as *Group Contribution Models (GCMs)* have been employed for predicting properties like vapor pressure. However, as datasets become more complex and high-dimensional, these traditional techniques are less effective in handling large-scale data with numerous interacting variables. Machine learning (ML) has emerged as a flexible and data-driven solution, offering advanced models capable of processing large datasets and capturing complex, non-linear relationships between molecular features and thermodynamic properties.

Among ML techniques, *Reinforcement Learning (RL)* has recently gained prominence as a promising method for feature selection and process optimization. Unlike static feature selection methods, RL allows dynamic and adaptive exploration of feature spaces, making it particularly effective in high-dimensional environments. This chapter reviews key studies that explore ML and RL in the context of feature selection.

3.1 Predicting Thermodynamic Properties

Accurately predicting thermodynamic properties, such as vapor pressure, activity coefficients, and phase equilibria, has been a key focus in chemical engineering. The Authors in [5] introduced the *UNIFAC (UNIQUAC Functional-group Activity Coefficients)* method, which has been used for predicting phase equilibria based on group contribution methods. While UNIFAC has proven effective for many applications, it encounters limitations when dealing with highly complex molecules and large datasets.

Another important advancement came from *Pankow and Asher (2008)*, who developed the *SIMPOL.1* model for predicting the vapor pressures of multifunctional organic compounds. This model also relies on group contribution methods and has shown success in many cases. However, the accuracy of such models is sensitive to the selection and identification of functional groups that influence vapor pressure, making careful model calibration essential for improving predictions [12].

3.2 Reinforcement Learning for Feature Selection

Feature selection is an essential task in machine learning, especially in high-dimensional datasets where irrelevant or redundant features can degrade model performance. In thermodynamic models, such as SIMPOL.1, the selection of functional groups plays a critical role in accurately predicting vapor pressure. Reinforcement Learning (RL) has emerged as a promising approach for feature selection due to its ability to navigate complex search spaces efficiently and dynamically optimize the selected feature subsets. This chapter reviews RL

techniques used in feature selection, based on previous research, and discusses their relevance to thermodynamic modeling, particularly in optimizing the SIMPOL.1 method.

Traditional feature selection methods, such as filter, wrapper, and embedded methods, attempt to reduce dimensionality by selecting the most relevant features from a dataset [1, 8]. However, these methods often struggle with high-dimensional data, where the feature space is vast and the relationships between features are non-linear. Reinforcement Learning (RL) offers a more flexible approach to feature selection by framing it as a sequential decision-making process [13, 4]. In this process, an agent learns to optimize the selection of features over time based on feedback (rewards) from the environment, typically tied to the model's performance.

3.3 Markov Decision Process (MDP) for Feature Selection

In RL-based feature selection, the problem can be formulated as a Markov Decision Process (MDP), where the agent navigates through different states representing subsets of features, and takes actions (e.g., adding or removing features) to optimize the model's performance. The reward function in this case is often linked to the improvement in model accuracy.

In [13], the MDP framework was employed to address feature selection using Temporal Difference (TD) learning. The study involved both synthetic datasets and real-world data from domains such as healthcare and image classification. By dynamically adjusting the feature set through exploration and exploitation, the agent was able to identify feature subsets that improved model accuracy more efficiently than traditional methods. Their experiments demonstrated that RL-based methods could outperform conventional feature selection techniques in terms of both computational efficiency and prediction accuracy.

This approach highlights the potential of MDPs in navigating high-dimensional feature spaces, which is particularly relevant in fields like thermodynamics, where the selection of functional groups directly influences the accuracy of vapor pressure predictions.

3.4 Deep Q-Network (DQN) for High-Dimensional Feature Selection

Deep reinforcement learning (DRL), particularly the Deep Q-Network (DQN), is a popular technique for addressing large discrete state space. DQNs combine RL with deep neural networks to approximate the value functions associated with each state-action pair, allowing the agent in our case to efficiently handle large feature spaces and complex, non-linear relationships.

In [18] they applied DQN to feature selection in the context of multisourced data cleaning. Their study utilized real-world IoT data from domains such as smart cities, where datasets were large and contained noisy, redundant features. The DQN-based approach improved feature selection by dynamically learning which features were most relevant while discarding irrelevant ones. Their results showed improved model accuracy and better handling of high-dimensional data compared to traditional methods.

3.5 Q-Learning with Upper Confidence Bound (UCB) Exploration

One of the key challenges in reinforcement learning is balancing exploration (trying new actions) and exploitation (using the best-known actions). Q-learning with Upper Confidence Bound (UCB) exploration is a method that encourages exploration by selecting actions with high uncertainty, thus helping the agent avoid getting stuck in local optima.

In this study [4] applied Q-learning with UCB exploration, focusing on benchmark datasets in machine learning. Their results demonstrated that UCB exploration improved learning efficiency, allowing models to converge faster compared to traditional exploration strategies like epsilon-greedy.

UCB exploration is particularly useful in early-stage learning, where the agent has little knowledge of the environment, may making it applicable to high-dimensional feature selection problems like our.

3.6 Inverse Reinforcement Learning (IRL) for Learning from Historical Data

Inverse Reinforcement Learning (IRL) allows the agent to learn the reward function by observing expert behavior rather than having the reward function specified manually. This is useful in cases where it is difficult to manually specify the reward structure but easier to collect demonstration data.

In their survey, [2] reviewed the applications of IRL in fields such as robotics, and autonomous systems. Their findings highlighted the effectiveness of IRL in learning reward functions from historical data, enabling agents to replicate expert decision-making without manually defining the rewards. Experiments showed that IRL-based approaches improved model generalization and accuracy in domains where manually specifying the reward function was challenging. In this thesis, IRL can be used to learn from historical data, avoiding the need to retrain models repeatedly when selecting new feature subsets. This can reduce computational costs and improve the efficiency of feature selection, especially in cases where data is massive.

Chapter 4

Methodology

This chapter details the methodology used in this thesis to implement and train a reinforcement learning-based feature selection system for improving vapor pressure predictions using the SIMPOL.1 model. The chapter is organized into sections that describe the datasets and preprocessing, the reinforcement learning framework, and the experimental setup.

4.1 Datasets and Preprocessing

4.1.1 Datasets Overview

Two datasets¹ were utilized in this thesis: the Dortmund Database (DDB) and the DIPPR database. The DDB dataset contains 140,890 experimental measurements of vapor pressure for 13,221 different organic compounds at various temperatures. The DIPPR dataset includes 23,323 measurements for 1,003 compounds. Both datasets consist of temperature, pressure, and molecular structure information, represented in Simplified Molecular Input Line Entry System (SMILES) notation. Both datasets are given as shown in Table 4.1.

id	T_K	p_MPa	Name	CAS-RN	SMILES
0	299.18	0.017900	METHANOL	67-56-1	CO
1	302.58	0.021286	METHANOL	67-56-1	CO
2	220.05	0.086233	KETENE	463-51-4	C=C=O
3	118.30	0.001200	ETHYLENE	74-85-1	C=C

TABLE 4.1: Raw Data: Temperature, Pressure, and Molecular Information

4.1.2 Feature Extraction

Each organic compound in the datasets was processed to extract functional group representations. A total of 60 functional groups were identified as show in Table 4.3, and each group was mapped to four temperature-dependent values:

$$f(T) = \begin{pmatrix} \frac{1}{T} \\ 1 \\ T \\ \ln(T) \end{pmatrix}^T$$

For each functional group i , a scalar v_i represents its frequency in the molecule. The overall feature vector F for a molecule is then:

¹The datasets were obtained from the Thermodynamics department in **Maschinenbau und Verfahrenstechnik** at **RPTU**, where the datasets underwent preprocessing to remove all compounds that do not contain elements other than oxygen, carbon, and hydrogen

$$F = v \cdot f(T)$$

Table 4.2 illustrates the final structure for both datasets, represented as G_{x-y} , where x ranges from 0 to 59 and y ranges from 0 to 3. Here, x indicates the functional group number as shown in Table 6, while y refers to the feature number among the four features for each functional group according to Equation 2.2.

id	G0_0	G0_1	G0_2	G0_3	G1_0	G1_1	G1_2	G1_3	G2_0	G2_1	G2_2	G2_3	...	G59_0	G59_1	G59_2	G59_3	SMILES	log10_p_ATMPa
0	0.003945	1	253.5	5.535364	0.003945	1	253.5	5.535364	0	0	0	0	...	0.011834	3	760.5	16.606091	CC=O	-0.774502
1	0.003888	1	257.2	5.549854	0.003888	1	257.2	5.549854	0	0	0	0	...	0.011664	3	771.6	16.649562	CC=O	-0.685570
2	0.003831	1	261.0	5.564520	0.003831	1	261.0	5.564520	0	0	0	0	...	0.011494	3	783.0	16.693561	CC=O	-0.607777

TABLE 4.2: Processed Data: Extracted Features for Each Molecule Based on Temperature (T) from Raw Data.

4.1.3 Data Splitting

The dataset was divided into three subsets: training, validation, and testing. For the DDB dataset, 1,000 unique molecules were reserved exclusively for the testing set, ensuring they were not part of the training or validation sets, providing truly unseen data for final evaluation. Similarly, for the DIPPR dataset, 20 unique molecules were set aside for testing.

After determining the testing sets, 10% of the remaining data was allocated for the validation set. The validation set was used to compute the performance of the SIMPOL.1 model during training. This performance was a critical component in computing the reward signal for the reinforcement learning agent, guiding it to make optimal feature selections by evaluating how the chosen features impact the model’s performance in predicting vapor pressure.

Additionally, all molecules that appeared only once in either dataset were included in the training set to enhance the training data and ensure that the model could generalize across a wide range of molecular structures.

4.1.4 Data Normalization

Due to the wide range of feature values, standard normalization was applied to all selected features. This step applied during training on the selected feature subset and ensures that each feature has a mean of 0 and a standard deviation of 1, preventing features with larger magnitudes from dominating the learning process.

4.2 Reinforcement Learning Approach for Feature Selection

4.2.1 Problem Formulation

Feature selection was framed as a sequential decision-making process using reinforcement learning. The agent’s objective is to have the ability to traverse the massive feature space efficiently and select the most relevant subset of functional groups that maximized the SIMPOL.1 model’s performance in predicting vapor pressure.

4.2.2 Environment

The environment was built using the OpenAI Gym framework, designed specifically for the feature selection process.

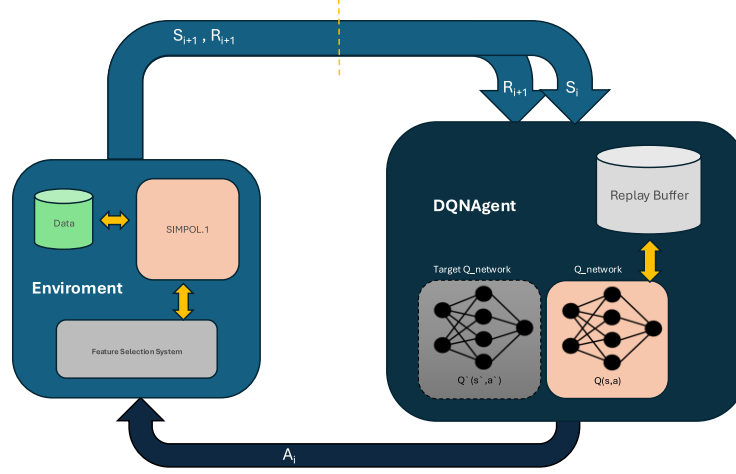


FIGURE 4.1: reinforcement learning approach

State Representation

At each time step, the state s_t is represented as a binary vector, where each element corresponds to a functional group in Table 4.3. If a group is selected, its corresponding entry is 1; otherwise, it is 0. This binary vector helps the agent keep track of which features are currently selected.

$$s_t = [s_t^1, s_t^2, \dots, s_t^n]$$

The state starts with all features selected (i.e., all entries are 1) to provide a baseline performance of the full model before features are removed.

Action Space

The action space includes $n + 1$ actions, where n is the number of functional groups. The agent can choose to deselect a specific group or select the "Stop" action, which ends the feature selection process. The action space is defined as:

$$\mathcal{A} = \{0, 1, \dots, n, \text{Stop}\}$$

4.2.3 Action Masking and Search Space Reduction

To enhance the learning process, action masking was implemented to prevent the agent from selecting redundant actions. Specifically, once a feature is deselected, the corresponding action is masked, reducing the search space by excluding actions that do not contribute to further improvement. This ensures that the agent focuses on meaningful decisions, avoiding wasted steps in each episode.

Additionally, the maximum number of steps per episode was set equal to the number of available actions (i.e., the number of functional groups). This step limit prevents episodes from running indefinitely, ensuring that the agent converges on an optimal subset of features more quickly. The process terminates either when the agent selects the "Stop" action or exhausts all valid actions. By combining action masking and limiting episode length, the

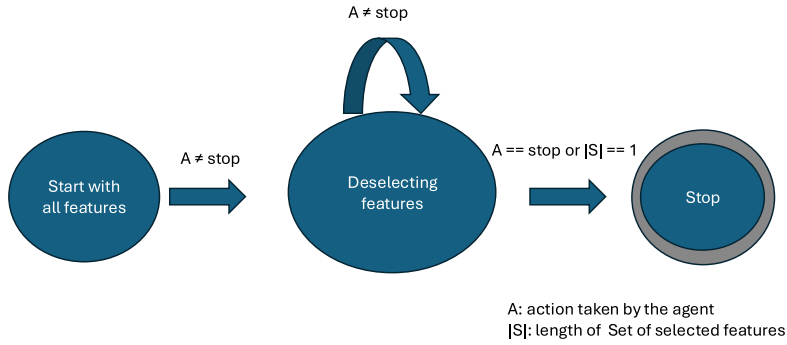


FIGURE 4.2: Feature Selection Mechanism

learning process becomes more efficient and structured, reducing computational complexity and speeding up convergence.

Reward Function

The reward function provides feedback based on changes in the model's (SIMPOL.1) performance on evaluation set. After each action a_t , the agent receives a reward r_{t+1} , calculated as the difference in performance (measured by Mean Squared Error (MSE) or Mean Absolute Error (MAE)) between the previous and current states:

$$r_{t+1} = \text{Performance}(s_t) - \text{Performance}(s_{t+1})$$

A positive reward is issued when the agent improves the model's performance, while a negative reward is given if performance worsens.

Reward Normalization: To avoid large differences in reward values that may arise from varying error scales across different states, I implemented a reward normalization technique. Normalizing the reward helps stabilize the learning process by reducing extreme reward fluctuations and makes it easier for the agent to interpret the reward signal effectively.

The reward is normalized using the following equation:

$$r'_{t+1} = \frac{r_{t+1}}{1 + |r_{t+1}|}$$

This technique ensures that the reward values are constrained between -1 and 1. When the error reduction is significant, the reward approaches 1, and when performance worsens, the normalized reward becomes closer to -1. This transformation makes it easier for the agent to learn from the reward signal, as it reduces the impact of extreme reward values, encouraging more stable learning over time.

4.2.4 Why Start with All Features?

Starting with all features allows the agent to remove irrelevant or redundant features by evaluating their impact on the model's performance. This approach ensures that the agent

has a clear understanding of the full model's performance as a baseline, which it can compare against as it removes features.

Another reason for following this strategy is related to the evaluation metrics used, such as MSE or MAE. If we were to start with no features and then add features one by one, the initial performance would be zero. A zero error implies that the initial state (with no features) gives the best performance, as the error is minimized, which does not make sense in the context of feature selection. It suggests that the empty feature set is the optimal solution, which is clearly misleading.

Additionally, starting with a large error for an empty set could make it harder for the agent to understand the relationship between features and the target variable. By starting with all features, the agent can quickly learn which features to deselect, shortening the time it takes to grasp the optimal feature subset and improving the learning efficiency.

4.2.5 Agent: Deep Q-Network (DQN)

The agent was implemented using two variations of the Deep Q-Network (DQN) to approximate the Q-value function $Q(s, a; \theta)$, where s is the state, a is the action, and θ represents the network parameters. The DQN predicts the expected cumulative reward for each action in the current state.

Policy: The policy $\pi(s)$ maps states to actions, and the Q-values were updated using the Bellman equation:

$$Q(s_t, a_t; \theta) \leftarrow Q(s_t, a_t; \theta) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) - Q(s_t, a_t; \theta) \right]$$

Two different network architectures were used to represent the Q-value function:

Standard DQN Model

The `DQNModel` is a simple feed-forward neural network composed of fully connected layers. The input to the network is the state vector, and the output is a set of Q-values, each representing the expected cumulative reward for taking a specific action in that state. The network structure consists of:

- An input layer with dimensionality matching the state space.
- Three hidden layers with dimensions of 128, 512, and 128, respectively.
- Rectified Linear Unit (ReLU) activations after each hidden layer.
- An output layer corresponding to the action space, predicting the Q-values for each possible action.

This architecture provides a basic yet effective model for learning the Q-values, where each Q-value estimates the long-term reward for taking a specific action.

Dueling DQN Model

The `DuelingDQNModel` improves upon the standard DQN by separating the Q-value function into two distinct streams:

- **Value stream:** This stream estimates the value of being in a given state, independent of the actions taken.

- **Advantage stream:** This stream estimates the advantage of each action, i.e., how much better it is to take a specific action compared to others in that state.

The final Q-value is computed by combining these two streams:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a') \right)$$

This architecture allows the network to better differentiate between the value of the state itself and the relative value of each action, which often improves learning efficiency in environments with many actions. The `DuelingDQNModel` consists of:

- Three hidden layers of 128, 256, and 256 units, respectively, shared between both streams.
- Two separate fully connected layers for the value and advantage streams.
- ReLU activations after each hidden layer.

The `DuelingDQNModel` generally provides better performance when the distinction between state value and action advantage is important [19].

4.2.6 Target Network and Soft Updates

A separate target network $Q(s_t, a_t; \theta^-)$ was used to stabilize the Q-value updates. In traditional Q-learning, the same Q-table is used to compute both the action-value estimates for the current state and the target for the update step. However, in Deep Q-Networks (DQN), this can lead to instability during training. This instability arises because neural networks are highly sensitive to updates, and when the same network is used to compute both the Q-values and the targets, the updates can be overly optimistic, causing the network to diverge or make inaccurate predictions.

The root of the problem is that when the weights of the network are updated during backpropagation, the target Q-values can shift unexpectedly, leading to inconsistent and unstable learning. To address this, a separate target network is introduced. This target network $Q(s_t, a_t; \theta^-)$ provides stable Q-value targets for the updates, as it is updated less frequently than the online network. This separation prevents the target values from changing too rapidly, improving the stability of the learning process.

The target network parameters θ^- were updated gradually using soft updates:

$$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$$

Where θ are the parameters of the online network, and τ is a hyperparameter that controls the rate of the update. A small value of τ , such as 0.01, ensures that the target network evolves slowly, providing consistent targets for the online network to learn from.

The reason for using soft updates, rather than hard updates (where the target network is updated all at once after a fixed number of steps), is to provide smoother and more stable learning. Hard updates can introduce sudden changes in the target network, which may lead to significant shifts in the Q-values, making the learning process less stable. By contrast, soft updates introduce gradual changes, helping the agent converge more reliably.

4.2.7 Exploration Strategies

Two exploration strategies were used:

- **Epsilon-Greedy Strategy:** Initially, the agent selects a random action with probability $\epsilon = 1$, encouraging exploration. Over time, ϵ gradually decays, eventually stabilizing at $\epsilon = 0.1$, shifting the agent's behavior towards exploiting its learned knowledge.
- **Upper Confidence Bound (UCB) Strategy:** The UCB strategy encourages exploration of uncertain actions by balancing exploitation and exploration through the formula:

$$UCB(s, a) = \hat{Q}(s, a) + c \cdot \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

where $\hat{Q}(s, a)$ represents the estimated Q-value for action a in state s , $N(s)$ is the total number of times state s has been visited, $N(s, a)$ is the number of times action a has been selected in state s , and c is a hyperparameter that controls the balance between exploration and exploitation. c is set to 2 to encourage broader exploration due to the large feature space of 60 features.

4.2.8 Replay Buffer

A replay buffer of size 100,000 was used to store experiences in the form of tuples $(s_t, a_t, r_{t+1}, s_{t+1})$. A mini-batch of size 128 was sampled during each update to break the correlation between consecutive samples, thus stabilizing training.

4.3 Optimization

The Adam optimizer was used to train the Q-network. The learning rate α was set to 0.001 after tuning, balancing between convergence speed and stability. Adam's adaptive learning rates provided efficient optimization, particularly for sparse gradients.

4.4 Experimental Setup

For the experiments I used Python, with the key libraries including PyTorch, Gym, NumPy, and RAPIDS for accelerated computing. The project was structured across various notebooks and scripts, each serving a specific role in the research. Data processing and feature extraction were performed interactively within Jupyter Notebooks, providing an organized environment for transforming the raw data into a usable form. Additionally, a separate notebook was dedicated to testing traditional feature selection methods such as Recursive Feature Elimination (RFE) and LASSO, where their performances with SIMPOL.1 were evaluated.

For the reinforcement learning (RL) framework, Python scripts were developed to handle the training and evaluation process, enabling smooth execution on external servers such as Elewe. To make these experiments easily configurable, I implemented a command-line interface (CLI), which allowed hyperparameters, such as learning rate, exploration strategies, and batch sizes and many more, to be modified without directly altering the script files. This streamlined the experimentation process, facilitating faster iterations and testing on various configurations.

The RL environment was built by extending the OpenAI Gym library, simulating the feature selection process for the SIMPOL.1 model. The environment interacts with the agent by providing states, actions, and rewards. It implements two key functions: `reset()` to initialize the environment with all features selected and `step()` to perform actions, return the next state, calculate rewards based on performance (Mean Squared Error or Mean Absolute Error), and indicate whether the episode has ended. The 'action_space' was defined as a

discrete space of $n + 1$ actions, where n represents the number of features, and the extra action allows the agent to signal the termination of feature selection. I also used the 'observation_space' to return the current state and action mask, guiding the agent's available decisions at each step.

Furthermore, to model the SIMPOL.1 process efficiently, I utilized Linear Regression from the Scikit-learn library without an intercept (setting 'intercept=False'). For faster computations, especially with large datasets, RAPIDS' implementation of Linear Regression running on a GPU was used to accelerate training.

4.4.1 Training Phases

The agent was trained with 100,000 episodes, gradually refining its ability to select optimal feature subsets. Initially, the agent relied heavily on exploration using the epsilon-greedy strategy, where ϵ started at 1, promoting a high level of randomness in action selection to explore the feature space. As training progressed, ϵ decayed over time and stabilized at 0.1, allowing the agent to shift from exploration to exploitation, focusing on the learned Q-values to make more informed decisions. The Adam optimizer, known for its adaptive learning rate capabilities, was employed with a learning rate of 0.001 to minimize the loss computed by the Mean Squared Error (MSE) between predicted and target Q-values.

To stabilize the learning process, a target network was used alongside soft updates with a factor $\tau = 0.01$. This approach prevents rapid changes in the target Q-values and helps the agent to learn more reliably. The replay buffer also played a crucial role by storing past experiences, allowing the agent to sample batches of experiences randomly during training. This technique reduces the correlation between consecutive actions and improves the generalization of the model. A batch size of 64 was used in the experiments, ensuring that the agent received sufficient variety in its training samples.

4.4.2 Evaluation Metrics

The performance of the models was evaluated using multiple metrics to ensure a better understanding of how well the agent was selecting features and how accurately the SIMPOL.1 model was performing vapor pressure predictions. The primary evaluation metrics were Mean Squared Error (MSE) and Mean Absolute Error (MAE), which were computed on both the validation and testing datasets. These metrics helped track the agent's ability to reduce the prediction error and identify the most relevant features while discarding irrelevant ones.

- **Mean Squared Error (MSE):** Measures the average squared difference between actual and predicted values. It is highly sensitive to outliers and penalizes larger errors more than smaller ones.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **Mean Absolute Error (MAE):** Measures the average absolute difference between actual and predicted values, providing a more interpretable metric for model accuracy as it directly reflects the average error in the same units as the data.

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

id	name	id	name
G1	Basic group	G31	Terminal alkyne (-CCH)
G2	Methyl group (-CH ₃)	G32	Internal alkyne (-CC-)
G3	Methylene group (-CH ₂ -)	G33	Double bond (-C=C-)
G4	Methine group (-CH-)	G34	Diol (-CH ₂ OH)
G5	Quaternary carbon (-C-)	G35	Alcohol attached to a methylene (-CH ₂ OH)
G6	Vinyl group (-CH=CH-)	G36	Methylenic group (-CH ₂ - in ring)
G7	Ethylene group (-CH=CH-)	G37	Methinic group (-CH- in ring)
G8	Terminal alkene (-CH=CH ₂)	G38	Quaternary carbon in ring (-C- in ring)
G9	Substituted alkene (-CH=CR-)	G39	Alcohol attached to methine (-CHOH)
G10	Aromatic carbon (Ar-H)	G40	Alcohol attached to quaternary carbon (-COH)
G11	Substituted aromatic carbon (Ar-R)	G41	Alcohol attached to ring (-CH ₂ OH in ring)
G12	Aromatic carbon with methyl group (Ar-CH ₃)	G42	Alcohol attached to ring (repeated entry)
G13	Aromatic carbon with methylene group (Ar-CH ₂ -)	G43	Keto-enol tautomer (enol form)
G14	Aromatic carbon with methine group (Ar-CH-)	G44	Ethylene group (-CH=CH-)
G15	Hydroxyl group (-OH)	G45	Internal alkyne (-CC-)
G16	Alcohol (-CH ₃ OH)	G46	Carbon monoxide (CO)
G17	Water (H ₂ O)	G47	Carbon dioxide (CO ₂)
G18	Phenol (Ar-OH)	G48	Methane (CH ₄)
G19	Acetyl group (-COCH ₃)	G49	Oxalate ion ([O=C=O] ²⁻)
G20	Ketone (-COCH ₂ -)	G50	Epoxide ring (oxirane)
G21	Carbonyl group (>C=O)	G51	Epoxide ring (substituted)
G22	Ester (-COOCH ₃)	G52	Epoxide ring (disubstituted)
G23	Ester (-COO-)	G53	Epoxide ring (symmetrical)
G24	Carboxylate anion (-COO-)	G54	Epoxide ring (asymmetrical)
G25	Methoxy group (-OCH ₃)	G55	Epoxide ring (fully substituted)
G26	Alkoxy group (-OCH ₂ -)	G56	Peroxy linkage (-O-O-)
G27	Ether (-O-)	G57	Propyl alcohol (CH ₃ CH ₂ CH ₂ OH)
G28	Carboxylic acid (-COOH)	G58	Isopropyl alcohol ((CH ₃) ₂ CHOH)
G29	Carboxylate (-COO-)	G59	Ethylene glycol (HOCH ₂ CH ₂ OH)
G30	Diol (-OH-CH ₂ -CH ₂ -OH)	G60	General carbon or oxygen atom

TABLE 4.3: Table of Functional Groups and Their Corresponding Names

Chapter 5

Results

This chapter presents the experimental results for feature selection using RL, alongside traditional feature selection methods such as Lasso, Recursive Feature Elimination (RFE), and Genetic Algorithms (GA). The datasets used for evaluation include the DDB and DIPPR datasets. I analyze the results using Mean Squared Error (MSE) and Mean Absolute Error (MAE), the number of selected features, and the computational efficiency of each method. Additionally, I assess the impact of acceleration techniques on training the SIMPOL.1 model, achieving around 6x speedup in computation time.

5.1 Traditional Feature Selection Methods Performance

I evaluated the performance of traditional feature selection methods (Lasso, RFE, and GA) on both the DDB and DIPPR datasets. The results in terms of MSE, MAE, and the number of selected features are summarized in Table 5.1.

Method	DDB			DIPPR		
	MSE	MAE	No. of Features	MSE	MAE	No. of Features
SIMPOL.1	0.08970	0.1485	60	0.0322	0.1520	60
Lasso	0.08972	0.1484	51	0.0325	0.1531	49
RFE	0.08970	0.1485	53	0.0247	0.1349	48
GA	0.07885	0.1553	34	0.0158	0.1044	31

TABLE 5.1: Performance of Traditional Feature Selection Methods for Both DDB and DIPPR Datasets for enhancing SIMPOL.1 method's performance

5.2 RL-Based Feature Selection Performance

I then evaluated the performance of the RL-based methods, specifically DQN and Dueling DQN, on the same datasets. The results are presented in Table 5.2.

Model	DDB Database			DIPPR Database		
	MSE	MAE	No. of Features	MSE	MAE	No. of Features
DQN	0.0733	0.1473	27	0.0194	0.1380	34
Dueling DQN	0.0726	0.1467	30	0.0192	0.1378	37

TABLE 5.2: Performance of RL-Based Feature Selection Methods for Both DDB and DIPPR Datasets

5.3 Impact of Exploration Strategy

The exploration strategy plays a important role in the learning process of the RL models. The models were trained using no exploration(greedy), Epsilon-Greedy and Upper Confidence Bound (UCB) strategies. Figure 5.1 shows the reward evolution over time for exploration strategies across both datasets.



FIGURE 5.1: Convergence of RL Models using Epsilon-Greedy and UCB Strategies for DDB and DIPPR Datasets

The UCB strategy demonstrated faster convergence and better overall performance than the Epsilon-Greedy approach for both datasets.

5.4 Computational Efficiency

I assessed the computational efficiency of the SIMPOL.1 model by implementing optimizations such as caching and GPU utilization through RAPIDS. These enhancements resulted in a 6x reduction in the agent’s training time compared to the non-accelerated version. Table 5.3 summarizes the computational efficiency of the RL models, showing the training time with and without acceleration techniques.

Dataset	Training Time	
	No Acceleration	With Acceleration
DDB	32 hours	5 hours
DIPPR	14 hours	2.5 hours

TABLE 5.3: Training Time with and without Acceleration for DDB and DIPPR Datasets

As seen in Table 5.3, the acceleration techniques applied to the SIMPOL.1 model reduced training time from several hours to mere minutes, reflecting a speedup of up to 6x. This improvement made the feature selection process more computationally efficient, especially when dealing with large datasets and high-dimensional feature spaces.

Method	DDB			DIPPR		
	MSE	MAE	# Selected Groups	MSE	MAE	# Selected Groups
Traditional Methods						
SIMPOL.1	0.0897	0.1485	60	0.0322	0.1520	60
Lasso	0.0897	0.1484	51	0.0325	0.1531	49
RFE	0.0897	0.1485	53	0.0247	0.1349	48
GA	0.0789	0.1553	34	0.0158	0.1044	31
RL-Based Methods						
DQN	0.0733	0.1473	27	0.0194	0.1380	34
Dueling DQN	0.0726	0.1467	30	0.0192	0.1378	37

Chapter 6

Discussion

The experimental results show that using Reinforcement Learning (RL) for feature selection in predicting vapor pressure with the SIMPOL.1 model can significantly improve the model's efficiency. Compared to traditional methods like Lasso, Recursive Feature Elimination (RFE), and Genetic Algorithms (GA), RL-based models achieved higher accuracy with fewer selected features and enhanced computational efficiency.

6.0.1 Impact on Prediction Accuracy

The RL-based models demonstrated a notable improvement in both accuracy and feature reduction across most experiments. For instance, the Dueling DQN model, which selected around 30 features, consistently outperformed traditional methods like Lasso, which used over 50 features in some cases. In the DDB dataset, the Dueling DQN achieved a Mean Squared Error (MSE) below 0.074, while traditional methods like Lasso and GA hovered between 0.078 and 0.089, requiring more features to achieve similar accuracy.

It was observed that Lasso did not provide any significant improvement for the SIMPOL.1 model, while the Genetic Algorithm showed better performance than RL on the DIPPR dataset. This may be due to the fact that RL requires more time to find the optimal solution in certain cases.

This reduction in the number of features while maintaining or even improving accuracy highlights the RL models' ability to capture complex, non-linear relationships between functional groups and vapor pressure. By contrast, traditional methods struggled to balance feature reduction with accuracy, often leading to higher errors or unnecessary complexity.

6.0.2 Exploration Strategies

The choice of exploration strategy also played a crucial role in the success of the RL models. The Upper Confidence Bound (UCB) strategy, in particular, led to faster convergence and better feature selection. As reflected in the reward evolution curves, UCB exploration allowed the RL agent to quickly focus on the most promising features, avoiding inefficiencies often encountered with epsilon-greedy strategies. This enabled models like Dueling DQN to reach lower error rates more rapidly, reducing both the MSE and the number of selected features faster than traditional exploration methods.

6.0.3 Computational Efficiency

The RL-based approach not only improved accuracy but also reduced computational time. With GPU acceleration and caching, the training time for the DDB dataset was reduced from 32 hours to just 5 hours, and for the DIPPR dataset, from 14 hours to 2.5 hours. This represents a significant speedup compared to traditional methods, which can be resource-intensive due to the need for multiple iterations of feature evaluation. However, when

using the GPU, there may be slight differences in evaluating the performance of SIMPOL.1 across different processors due to precision issues, though these differences should be minor.

This efficiency, coupled with the RL models' ability to select fewer, more relevant features, demonstrates the practical advantages of using RL for feature selection, especially in high-dimensional datasets where time and resources are limited.

6.0.4 Limitations and Future Work

While the RL-based methods performed well in this study, there are some limitations. The initial setup of the RL models, including tuning hyperparameters and selecting the right exploration strategy, required significant computational resources, especially when increasing the batch size. Additionally, while the models excelled on the datasets used in this project, further work is needed to explore their applicability to other datasets and properties beyond vapor pressure.

Future research could focus on further optimizing RL algorithms, such as statistical models, and collecting more historical data for applying Inverse Reinforcement Learning (IRL). This could be achieved by gathering data from the agent as it explores the environment. Moreover, applying RL-based feature selection in other scientific fields could uncover broader applications for this method.

Chapter 7

Conclusion

In this thesis, I explored how reinforcement learning (RL) can be used for feature selection in predicting vapor pressure with the SIMPOL.1 model. By treating feature selection as a decision-making process, RL showed itself to be a flexible approach that can optimize the selection of functional groups, resulting in more accurate predictions while keeping the model simpler.

The experiments showed that RL models, especially Dueling Deep Q-Networks (Dueling DQN), performed better than traditional methods like Lasso, Recursive Feature Elimination (RFE), and Genetic Algorithms (GA). RL achieved lower Mean Squared Error (MSE) and Mean Absolute Error (MAE) using fewer features, which means it was able to handle the complex interactions between functional groups and vapor pressure more effectively. The use of exploration strategies like Upper Confidence Bound (UCB) also helped make the feature selection process faster and more efficient.

Moreover, the computational efficiency of RL was improved by using techniques like GPU acceleration and caching, which reduced the training time by up to six times in some cases. This makes RL a practical option when dealing with large datasets.

However, there were some challenges. Training RL models required a lot of computational resources, and tuning the hyperparameters took time. While the results were promising on the datasets used in this project, more work is needed to see if RL can be applied successfully to other datasets and properties beyond vapor pressure.

In conclusion, this thesis shows that reinforcement learning can be a useful tool for feature selection, potentially outperforming traditional methods. It offers a way to handle complex, high-dimensional datasets while improving predictive accuracy. Future work could focus on improving RL more advanced algorithms like PPO and exploring different ways of representing the environment and state like Graphs in order to enhance the feature selection process.

Bibliography

- [1] Mohamed Abd Elaziz et al. "Feature Selection for High Dimensional Datasets Based on Quantum-Based Dwarf Mongoose Optimization". In: *Mathematics* 10.23 (2022), pp. 1–19.
- [2] Saurabh Arora and Prashant Doshi. "A survey of inverse reinforcement learning: Challenges, methods and progress". In: *Artificial Intelligence* 297 (2021), p. 103500.
- [3] Avrim L Blum and Pat Langley. "Selection of relevant features and examples in machine learning". In: *Artificial intelligence* 97.1-2 (1997), pp. 245–271.
- [4] Kefan Dong et al. "Q-learning with ucb exploration is sample efficient for infinite-horizon mdp". In: *arXiv preprint arXiv:1901.09311* (2019).
- [5] Jürgen G Gmehling, Thomas F Anderson, and John M Prausnitz. "Solid-liquid equilibria using UNIFAC". In: *Industrial & Engineering Chemistry Fundamentals* 17.4 (1978), pp. 269–273.
- [6] Isabelle Guyon and André Elisseeff. "An introduction to variable and feature selection". In: *Journal of machine learning research* 3.Mar (2003), pp. 1157–1182.
- [7] Kevin G Joback and Robert C Reid. "Estimation of pure-component properties from group-contributions". In: *Chemical Engineering Communications* 57.1-6 (1987), pp. 233–243.
- [8] Alan Jović, Karla Brkić, and Nikola Bogunović. "A review of feature selection methods with applications". In: *2015 38th international convention on information and communication technology, electronics and microelectronics (MIPRO)*. Ieee. 2015, pp. 1200–1205.
- [9] Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. "Reinforcement learning: A survey". In: *Journal of artificial intelligence research* 4 (1996), pp. 237–285.
- [10] Ron Kohavi and George H John. "Wrappers for feature subset selection". In: *Artificial intelligence* 97.1-2 (1997), pp. 273–324.
- [11] Volodymyr Mnih et al. "Human-level control through deep reinforcement learning". In: *nature* 518.7540 (2015), pp. 529–533.
- [12] J. F. Pankow and W. E. Asher. "SIMPOL.1: a simple group contribution method for predicting vapor pressures and enthalpies of vaporization of multifunctional organic compounds". In: *Atmospheric Chemistry and Physics* 8.10 (2008), pp. 2773–2796. DOI: 10.5194/acp-8-2773-2008. URL: <https://acp.copernicus.org/articles/8/2773/2008/>.
- [13] Sali Rasoul, Sodiq Adewole, and Alphonse Akakpo. "Feature selection using reinforcement learning". In: *arXiv preprint arXiv:2101.09460* (2021).
- [14] RC Reid. *The properties of gases and liquids*. 1987.
- [15] Joseph Mauk Smith. *Introduction to chemical engineering thermodynamics*. 1950.
- [16] Richard S Sutton. "Reinforcement learning: An introduction". In: *A Bradford Book* (2018).

- [17] Hado Van Hasselt, Arthur Guez, and David Silver. "Deep reinforcement learning with double q-learning". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 30. 1. 2016.
- [18] Qianlong Wang et al. "Deep Q-network-based feature selection for multisourced data cleaning". In: *IEEE Internet of Things Journal* 8.21 (2020), pp. 16153–16164.
- [19] Ziyu Wang et al. "Dueling network architectures for deep reinforcement learning". In: *International conference on machine learning*. PMLR. 2016, pp. 1995–2003.
- [20] Christopher JCH Watkins and Peter Dayan. "Q-learning". In: *Machine learning* 8 (1992), pp. 279–292.